Selecting Reliable Mathematical Reasoning with Arithmetic Evidence

DASE7506 Advanced Machine Learning Methods | Mini Project 2
28 October 2026

Luo Xinqiu | Student ID: 3036707989

Abstract

This project studies whether inexpensive evidence from written calculations can improve the selection of a reliable answer among four supplied reasoning traces. The final method combines embedding-based predictors with gradient-boosted trees over candidate metadata, arithmetic consistency, dependency statistics and cross-candidate agreement after numerical recomputation. All fitted parameters use the supplied training split. Methods are shortlisted using question-level out-of-fold predictions, selected on official validation data, and frozen before test scoring. The final checkpoint achieves 393/467 correct on the complete test bank (84.1542%), compared with 361/467 for the reproduced weighted MLP baseline. A fresh reproduction returns exactly the same 467 selections in 1.822 seconds on a four-thread Apple M5 CPU configuration. Arithmetic process features provide the clearest internal-development gain; the final repair-feature increment has mixed evidence across splits. The results support using arithmetic checks as learned evidence, while exposing limits of semantic interpretation and repeated development-set selection.

1 Task and evaluation protocol

Each question provides four candidate texts and fixed 1024-dimensional embeddings. The method returns one candidate index. The official parser and scorer award credit for a matching final numeric answer, even when the reasoning contains mistakes. Several candidates can be correct. Question IDs serve only for alignment and are not model features. The supplied data and evaluation tools are kept unchanged [1].

Table 1. Data splits and difficult subsets

Split	Questions	Answer disagreement	Single correct
Train	4,755	2,114	711
Validation	798	355	117
Test	467	467	Not analyzed

Disagreement means at least two distinct valid parsed answers. Single-correct means exactly one candidate matches the reference. Train and validation counts are computed with the fixed official parser; no test subgroup or error analysis was used.

Train contains 2,242 all-correct questions and validation contains 379. Such questions are much easier to score than selecting the unique correct answer. Overall validation accuracy therefore overstates performance on difficult cases. Test consists entirely of disagreement questions, so validation-disagreement accuracy is a more relevant development metric than the unrestricted total.

The resource constraints are 16 MiB of uncompressed additional inference assets and 30 seconds for a fresh CPU prediction process plus an independent scoring process, with four threads in total. The reference CPU is an Intel Xeon Platinum 8457C. Local timings do not establish performance on that reference machine [1].

2 Method

2.1 Weighted candidate supervision

A candidate label is one when its parsed final answer equals the parsed reference. The baseline is a 1024-to-32-to-1 ReLU MLP trained with binary cross-entropy. All four candidates from a question remain in the same split. To reduce domination by easy all-correct groups, the question weights for one, two, three and four correct candidates are respectively 2.0, 1.5, 1.0 and 0.25. These weights use training labels only. The loss averages candidate binary cross-entropy within each question and weights the resulting question loss. The final ensemble retains three MLP seeds, 17, 29 and 43, trained for 16 epochs with L2 coefficient 0.01. Standardization statistics are fitted on the corresponding training portion.

2.2 Input evidence

Table 2. Candidate feature blocks

Feature block	Dimensions	Contents
Fixed embeddings	1,024	Supplied representation of each candidate
Metadata	116	Length, lexical cues, parsed answer, votes, simple equations and relative values
Process evidence	212	Numeric equation consistency, coverage, dependencies, endpoints and cross-candidate values
Repair evidence	116	Recomputed values, propagation, final-answer agreement and cross-candidate support

Each handcrafted block contains raw descriptors and their differences from the mean across the four candidates. This compares candidates without encoding their positions. The process-only tree uses 328 dimensions (metadata plus process); the repair-full tree uses all 1,468 dimensions. Metadata and process checks overlap, so these dimensions should not be interpreted as independent pieces of evidence.

The arithmetic extractor normalizes common multiplication/division symbols, percentages and simple LaTeX fractions. A restricted abstract syntax tree accepts numerical constants, parentheses and addition, subtraction, multiplication or division. Unsupported algebra and prose are skipped; arbitrary text is never executed. Extracted equations supply counts and magnitudes of numerical inconsistencies, coverage of numbers from the question, whether intermediate values are reused, and whether the final answer appears at an equation endpoint.

2.3 Recomputing stated calculations

Three input-only strategies recompute existing expressions. Local evaluates each expression independently. Protected propagates corrections to later occurrences of a stated intermediate value, while protecting question numbers and common constants. Permissive allows broader propagation. For example, a trace stating 84 + 21 + 5 = 100 and then 420 - 100 = 320 can yield 310 after propagation. This is an illustration, not an additional training example. The value 100 may also denote an unrelated percentage base, which makes unrestricted numerical substitution ambiguous.

The repair block measures whether corrected values support any of the four supplied answers, how often values change, and whether corrections propagate. It never creates a fifth candidate or forces a repaired answer to win. The new repair path strips line-start list markers before parsing. Existing process features retain their frozen parsing behavior for compatibility.

3 Ensemble and model selection

3.1 Frozen predictor

CatBoost classifiers provide nonlinear interactions among numerical features [2]. The principal trees use 800 iterations, depth 5, learning rate 0.05, leaf L2 regularization 5, 64 borders and seed 17. Models are trained with four CPU threads. Tree logits are combined with the MLPs and a small kernel classifier. The kernel uses training-fitted, whitened PCA with 128 components, 512 Nyström landmarks, an RBF kernel with gamma 0.25/128, and logistic regression with C = 0.1 on valid candidates from disagreement questions. The Nyström API constructs an approximate kernel feature map [3]; its fitted transformation is folded into exported NumPy coefficients for inference.

For each component, its four logits are centered within the question and divided by a fixed scale estimated from its Train OOF logits. Weighted normalized scores are summed. Candidates with unparseable answers receive a large negative score, and the highest-scoring candidate is selected. Centering uses only the current question; the fixed scale requires no evaluation labels. If all answers are unparseable, the implementation falls back to the first tied index.

Table 3. Frozen ensemble composition

Frozen component	Weight	Feature input
Repair-full tree	0.25000	Embedding + metadata + process + repair
Process tree, first stored copy	0.18750	Metadata + process
Process tree, second stored copy	0.28125	Metadata + process
PCA and RBF kernel classifier	0.05625	Fixed embeddings
Original full tree	0.11250	Embedding + metadata
Three MLPs	0.03750 each	Standardized embeddings

Weights sum to one. The two process-tree copies have identical exported trees and OOF scores. Their effective combined weight is 0.46875, so they do not provide independent ensemble diversity. Both files remain in the frozen bundle to reproduce it exactly. Component scales are stored in config.json.

3.2 Development decisions

The initial E26-E28 study used a 3,804-question internal development portion and a 951-question confirmation set. Its proposed MLP ensemble did not pass the predeclared confirmation criterion, so the baseline was retained. From E29 onward, experiments used three question-level Train folds with seed 7306, stratified by correctness count and disagreement. Preprocessing and model fitting respect fold boundaries; training data alone determines fitted parameters.

Each experiment locks a limited shortlist from Train OOF results before evaluating it on official validation. The final repair experiment uses six training specifications, snapshots at 300 and 800 trees, a two-seed average, and fusion weights 0.25, 0.5, 0.75 and 1.0. This gives 57 scored recipes sharing 18 CV fits. The shortlist contains the best three distinct feature families, the best standalone model and the incumbent. Validation selection prioritizes disagreement correct, then overall correct, then single-correct; exact ties prefer the incumbent. Source and assets are hashed before final testing.

4 Quantitative results

Table 4 reports selected checkpoints from the retained development sequence. Test columns are historical measurements of frozen versions, not scores used to rank the shortlisted models. E34 and E38 retained their parent and inherited its existing test result without another test run. All rows use the same official banks, but they represent sequential development rather than independent randomized trials.

Table 4. Selected checkpoints on official banks

Stage and selected method	Val /798	Test /467	Test %
E28 Weighted MLP baseline	722	361	77.3019
E32 Full tree plus three MLPs	745	375	80.2998
E34 Answer-group study, parent retained	745	375*	80.2998*
E36 Kernel blend	745	375	80.2998
E38 Context study, parent retained	745	375*	80.2998*
E40 Process-evidence blend	747	388	83.0835
E42 Process-weight adjustment	748	389	83.2976
E44 Repair-evidence blend	749	393	84.1542

**Inherited score, not a new test evaluation. The E28 legacy baseline does not use the later unparsed-answer mask; therefore the end-to-end improvement includes decoder and model changes and is not a pure feature ablation.*

Relative to E28, E44 gains 32 test questions, or 6.8522 percentage points. The largest later step is E40, which adds 13 test questions relative to E36. This comparison is descriptive: it neither isolates every component nor eliminates selection effects from the development process.

Table 5. Final model against its immediate parent

Subset	E42 parent	E44 final
Train OOF overall	4,385/4,755 (92.2187%)	4,380/4,755 (92.1136%)
Train OOF disagreement	1,744/2,114 (82.4976%)	1,739/2,114 (82.2611%)
Train OOF single-correct	393/711 (55.2743%)	386/711 (54.2897%)
Validation overall	748/798 (93.7343%)	749/798 (93.8596%)
Validation disagreement	305/355 (85.9155%)	306/355 (86.1972%)
Validation single-correct	74/117 (63.2479%)	75/117 (64.1026%)

The final repair increment is not consistently positive across development splits. Its Train OOF disagreement score drops by five questions and its single-correct score drops by seven, while validation improves by one on each metric. The final model is the validation winner within the locked family shortlist, not the best Train OOF recipe. This distinction prevents attributing a stable generalization gain to the repair mechanism alone.

On 28 October 2026, the unchanged E44 checkpoint was rerun through the official prediction and independent scoring programs. It again achieved 393/467 (84.1541755888651%), with all 467 candidate indices identical to the previous frozen output. No model selection or parameter changes occurred during this reproduction.

5 Ablations and strength of evidence

5.1 Which additions helped

The process-evidence study gives the strongest internal comparison: the selected blend improved Train OOF disagreement from 1,700 to 1,744 correct, with fold changes of +23, +10 and +11. It corrected 105 previous errors while introducing 61 errors. The descriptive paired-bootstrap interval was +0.8515 to +3.2640 percentage points. Official validation improved from 745 to 747 correct. Repeated use of the development folds still limits the inferential strength of these numbers.

Several alternatives failed to displace the incumbent. Grouping equivalent answers and pooling their features reached at most 743/798 for new validation candidates in E33-E34. Additional question-context features in E37-E38 reached 745/798 but had fewer single-correct successes than the incumbent. E41-E42 varied seeds, depth and PairLogit ranking. The selected seed-17 tree reproduced the old process tree exactly; its gain was a changed fusion weight, not new diversity from seed ensembling.

5.2 Final repair shortlist

Table 6. Final repair shortlist

Recipe added to E42	OOF hard /2114	Val /798	Val single /117
Two repair seeds, 300 trees, weight .25	1,748	748	73
Repair-full, 800 trees, weight .25	1,739	749	75
Repair-only, 300 trees, weight .25	1,733	745	71
Standalone repair seed 17, 800 trees	1,714	739	65
E42 incumbent	1,744	748	74

The final selected recipe changes -1, -3 and -1 disagreement outcomes across the three Train folds. It gains 18 and loses 23 questions; its paired-bootstrap 95% interval is -0.8042 to +0.3784 percentage points. The interval crosses zero. These intervals are descriptive and do not adjust for searching many correlated recipes. On validation, the final increment gains three questions and loses two, leaving 47 questions wrong under both models.

5.3 Why repair remains a feature

Table 7. Recomputation diagnostics on Train

Recomputation strategy	Changed values	Wrong to gold	Correct to wrong
Local	222	108	17
Protected	284	150	18
Permissive	337	160	51

Train candidate-level diagnostics using relative numerical tolerance 1e-5. These count changed final-value proposals, not official selection accuracy. Values need not be available as candidate answers.

Permissive propagation fixes more values but also damages more correct ones. In the older process extraction, 522 candidates with correct final answers contain a detected inconsistent equation, compared with 566 among candidates with wrong final answers. A detector flag can reflect a genuine intermediate error or a parsing artifact. A hard veto would therefore reject some successful candidates. The repair-full winner also includes embeddings and older features, so its result does not isolate repair from all other inputs.

6 Qualitative analysis and limitations

The following examples come only from the saved official-validation comparison between E42 and E44. They explain observed gains and losses after selection; they were not turned into lookup rules. Their presence among changed decisions does not prove that a particular feature caused the change.

A missing term in a correct calculation

A costume requires three skirts, each using 12 by 4 square feet, plus a bodice using 2 + 2 x 5 square feet. At 3 dollars per square foot, the total is (144 + 12) x 3 = 468. The earlier selection computed the bodice area but omitted it from the final cost, returning 432. E44 selects 468. Every local multiplication in the wrong path can be correct; the error is failure to carry a relevant quantity into the final expression. Coverage and dependency evidence can be useful here, but it is not a full semantic proof.

A multiplication error propagated to the endpoint

For the population question, 15 x 58,940 equals 884,100. After 130,000 leave, half of the remaining population is 377,050. The earlier chosen trace used 883,100, producing 376,550. E44 instead selects the candidate returning 377,050. This is the kind of numerical propagation error that motivates recomputation of stated expressions. Both the incorrect and correct traces are fluent, so text length or confidence alone is insufficient.

Losses despite explicit numerical evidence

For five concert tickets, the correct total is 100 + (2 x 2.4 x 50 - 10) + 30 = 360. E42 selected 360, whereas E44 selects a trace that incorrectly simplifies the scalper cost and returns 320. A second loss confuses revenue with profit: selling 28 sweaters at 35 dollars with yarn cost 4 x 6 dollars per sweater yields profit 308, but the final selection returns revenue 980. These examples show that a learned ensemble can still ignore arithmetic warnings or choose the wrong target quantity.

Protocol and uncertainty

Formatting and the official parser affect the labels: a human-readable final value is not always the value extracted from a complete trace. The scorer remains fixed, and the implementation masks only candidates it cannot parse. The report evaluates the specified selection task rather than certifying all reasoning steps. Safe arithmetic extraction does not solve entity grounding, implicit equations, rounding conventions or the distinction between costs, revenues and profits.

Validation has been consulted across multiple stages and the same Train folds have been reused. The final validation advantage is only one question over E42, and its OOF trend goes the other way. Several frozen versions have also been reported on the public test bank during development. Consequently, the reported test history is not equivalent to a single untouched final evaluation. Test references were not used for fitting, hyperparameter selection or per-question error mining. A separate audit bank is needed to assess generalization beyond this public bank.

The evidence supports arithmetic process features as complementary predictors under tight resource limits. It does not establish that more repair propagation, a larger ensemble, or a higher public score will generalize monotonically. Future changes should be selected using new internal evidence and evaluated on an untouched audit set, rather than tuned to known test outcomes.

7 Resources and reproducibility

The submitted inference assets total 4,053,732 file bytes. Counting uncompressed NumPy entries and complete native CatBoost binaries plus configuration gives 4,050,842 bytes (3.863 MiB); a more conservative JSON-tree plus NumPy representation totals 11,666,720 bytes (11.126 MiB). Both are below 16 MiB. All nine files in the final directory are required, including both stored process trees and config.json. Training caches and diagnostic JSON files are not loaded at inference.

The fresh reproduction on an Apple M5 MacBook Air with 24 GB memory took 1.764587 seconds for the complete prediction process and 0.057747 seconds for the independent scorer, totaling 1.822334 seconds. This includes startup, imports, loading and preprocessing. CPU libraries were restricted to four threads; no GPU was used. The reference Xeon was not measured. Internal program timing fields are recorded separately and are not substituted for full-process wall time.

Table 8. Recorded search costs

Retained study	Recorded pipeline seconds	Work represented
E26-E28 MLP study	5.783	CV research wall time; confirmation costs separate
E29-E32 Expanded models	228.410	40 CV and full-Train fits
E33-E34 Answer groups	332.187	18 fits
E35-E36 Kernel methods	9.859	134 PCA, mapping and classifier cost events
E37-E38 Context features	500.709	21 fits
E39-E40 Process features	249.569	21 fits
E41-E42 Stability and objectives	202.992	32 successful fits
E43-E44 Repair features	151.013	22 fits

Recorded pipeline times sum to 1,680.522 seconds. This is not the total project compute cost: it excludes earlier prototypes, separately logged confirmation/diagnostic work, a failed E41 attempt (approximately 43.31 seconds inferred from file times), and discarded outputs. Kernel cost events are not all model fits, and scored fusion recipes share fitted models. Full logs and nine-column records preserve the distinction.

Environment: Python 3.12.0, NumPy 2.5.3, CatBoost 1.2.8, scikit-learn 1.7.2 and SciPy 1.16.3. The implementation commit is 7d633199073d36020b0298440fa7d753c846f05e. The E44 freeze file records SHA-256 hashes for source and all inference assets. Before and after this reproduction, both sets of hashes matched. Data checks passed and the regenerated prediction indices matched the original output exactly.

The retained implementation also passed singleton versus batch, question-order and candidate-order checks on recorded samples. These checks verify selected interface properties, not correctness for every possible audit input. The method consumes only the supplied bank, embeddings and explicit checkpoint; it does not open references or store a table of predictions indexed by question ID.

8 Reproduction commands and conclusion

Run from the repository code directory with Python 3.12 and the official data present. Use a fresh output directory. In the supplied final package, place the checkpoint folder under code/ or supply its absolute path. These commands reproduce a fixed model; they do not choose a new one.

```
python3.12 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
export OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4
export MKL_NUM_THREADS=4 VECLIB_MAXIMUM_THREADS=4
.venv/bin/python check_data.py
mkdir -p runs/e44-check
CHECKPOINT=runs/repair-20261027-235139/final
.venv/bin/python predict.py --method submission.py \
  --checkpoint "$CHECKPOINT" --split test \
  --output runs/e44-check/test-predictions.json
.venv/bin/python score_predictions.py --split test \
  --predictions runs/e44-check/test-predictions.json \
  --output runs/e44-check/test-score.json
```

To reproduce the final training stage, use the documented parent E42 Train OOF cache and frozen parent assets. The training-evidence archive preserves this dependency. The archived RESULTS.md also provides the complete expanded, kernel, process, stability and repair chain for rebuilding the parent models from supplied Train data.

```
mkdir -p runs/e44-retrain
.venv/bin/python train_scorer.py --repair \
  --run-dir runs/e44-retrain \
  --parent-run runs/stability-20261027-233342-retry
.venv/bin/python analyze_e22.py --diagnose-repair runs/e44-retrain
.venv/bin/python analyze_e22.py --select-repair runs/e44-retrain
```

The selection command evaluates the previously locked validation shortlist, freezes the selected files and then scores the new frozen checkpoint once. It inherits a verified parent result if the incumbent is retained. Existing started or scored run directories must not be overwritten. Reproduction timings may vary with machine load; a retrained CatBoost file can differ in metadata, so the supplied frozen assets are the direct score-reproduction target.

Conclusion

The final selector reproduces 84.1542% test accuracy with a small offline CPU ensemble. The most convincing gain comes from adding process-level numerical evidence to embedding and metadata predictors. Repair-derived agreement offers a modest validation-selected increment, but its internal-development evidence is mixed. Correct numerical operations alone cannot guarantee that a candidate solves the intended problem.

AI assistance and attribution

OpenAI Codex assisted with experiment design, source editing, execution, verification, analysis and report preparation. The project uses supplied GSM8K-derived questions and Qwen3-0.6B candidate traces and fixed representations; no additional model weights or external training data were used. CatBoost, NumPy, scikit-learn and SciPy are reused software. The supplied MIT license and data attribution files are retained. AI assistance is disclosed here as required by the coursework.

References and evidence

[1] DASE7506 MP2. Selecting Reliable LLM Reasoning. Supplied guide/GUIDE.md and code/README.md, protocol dase7506-mp2-open-hard-verifier-v5, Fall 2026.

[2] CatBoost documentation. CatBoostClassifier and prediction API. https://catboost.ai/docs/en/concepts/python-reference_catboostclassifier (accessed 28 October 2026).

[3] scikit-learn 1.7.2 documentation. Nystroem. https://scikit-learn.org/1.7/modules/generated/sklearn.kernel_approximation.Nystroem.html (accessed 28 October 2026).

[4] Local experiment artifacts. E28-E44 run directories; final-results.json, train-diagnostics.json, training-costs.json, repair-coverage.json and freeze.json. Final reproduction: runs/final-report-20261028/test-reproduction.json.